

Desenvolvimento de IA para Análise Preditiva

Mini-Projeto Avaliativo - Módulo 2 - Semana 05

SUMÁRIO

1 CONTEXTUALIZAÇÃO	2
2 DESAFIO	2
3 RESULTADOS ESPERADOS (ENTREGA)	2
4 REQUISITOS DA APLICAÇÃO	3
5 ROTEIRO DA APLICAÇÃO	4
5.1 FORMATO DO SISTEMA	4
5.2 DOCUMENTAÇÃO NO README.MD	5
5.3 USO DO GITHUB	5
5.4 GRAVAÇÃO DE VÍDEO	5
6 CRITÉRIOS DE AVALIAÇÃO	6
7 PLANO DE PROJETO	8
8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL	8

1 CONTEXTUALIZAÇÃO

A transição dos dados tabulares tradicionais para o processamento de dados não estruturados (como imagens, sinais e textos) representa um dos marcos mais importantes da Ciência de Dados moderna. O dataset MNIST (Modified National Institute of Standards and Technology) é o benchmark clássico da Visão Computacional, composto por 70.000 imagens em escala de cinza (28 x 28 pixels) de dígitos manuscritos de 0 a 9.

Neste projeto do Módulo 2, você atuará como Cientista de Dados e Engenheiro de Machine Learning, tendo liberdade de explorar o contraste entre algoritmos clássicos de aprendizado estatístico e redes neurais profundas (Deep Learning). O objetivo central é compreender como diferentes arquiteturas extraem representações dos dados, analisar criticamente métricas de desempenho multiclasse e testar a robustez e generalização extrema dos modelos diante de cenários adversos e dados do mundo real.

2 DESAFIO

Você deverá construir, treinar, validar, comparar e estressar um Pipeline Preditivo Multiclasse ponta a ponta capaz de classificar dígitos manuscritos a partir do dataset MNIST (mnist_784).

Exemplo de importação da base de dados MNIST:

Python

```
from sklearn.datasets import fetch_openml

print("Baixando o dataset MNIST...")

mnist = fetch_openml('mnist_784', version=1, as_frame=False,
                    parser='auto')

print("Finalizou o processo de baixar o dataset MNIST!")
```

- Scripts de apoio explicados em aula: [MiniProjeto_Modulo02](#)
- Projetos kaggle: [MNIST Dataset](#) (explorem pra ter uma ideia do que outros cientistas e estudantes já fizeram com essa base de dados).

O desafio é dividido em duas frentes fundamentais:

- **Benchmark Comparativo:** Implementar e contrastar o desempenho de **3 modelos preditivos distintos** (podendo mesclar entre algoritmos clássicos como SVM, Random Forest, KNN, Gradient Boosting e Redes Neurais Artificiais/MLP com TensorFlow/Keras).

- **Estresse e Robustez (Testes de Generalização Extrema):** Avaliar o comportamento dos modelos sob premissas fora da distribuição de treino (OOD - *Out-of-Distribution*) e testar a capacidade de inferência com dígitos manuscritos digitalizados pelos próprios alunos.

3 RESULTADOS ESPERADOS (ENTREGA)

O código deverá ser inserido e versionado no **GitHub em modo público**, o vídeo deverá ser inserido no **Google Drive em modo leitor para qualquer pessoa** com o link. Ambos os **links** deverão ser disponibilizados na tarefa **Módulo 2 - Mini-Projeto Avaliativo**, presente **na Semana 06 do módulo 02** do AVA até o dia **14/09/2026 às 22h**.

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. **Não esqueça de submeter os links no AVA (Vídeo e GitHub).** Os links não submetidos terão penalidade na nota.

4 REQUISITOS DA APLICAÇÃO

O desenvolvimento técnico do seu código Python no Notebook deve, obrigatoriamente, ser segmentado e documentado nas seguintes etapas estruturadas:

Fase 1: Carregamento e Análise Exploratória de Imagens (EDA)

- Importar o dataset MNIST via scikit-learn (`fetch_openml('mnist_784')`) ou via [tensorflow.keras.datasets.mnist](#).
- Exibir a dimensionalidade das matrizes (X e y) e comprovar a distribuição das classes (balanceamento de dígitos de 0 a 9).
- Gerar uma grade visual de no mínimo 2 x 5 com [matplotlib](#), exibindo exemplos de imagens de cada dígito junto com seus respectivos rótulos originais.
- Inserir célula de texto interpretando a estrutura dos dados: explicar o significado da escala de intensidade de pixels (0 a 255) e como imagens bidimensionais (28 x 28) são representadas vetorialmente (784 features).

Fase 2: Pipeline de Pré-processamento e Divisão dos Dados

- Divisão Estratificada: Dividir a base em Treino, Validação e Teste (ex: 70% treino, 10% validação e 20% teste ou 80% treino/validação e 20% teste), garantindo estratificação por classe (`stratify=y`).
- Normalização / Escalonamento: Aplicar o redimensionamento dos pixels para a escala [0.0, 1.0] (dividindo por 255.0 ou [via MinMaxScaler/StandardScaler](#)).
- Justificar textualmente a importância da normalização para a convergência de modelos lineares e distâncias métricas.

Fase 3: Implementação e Treinamento dos 3 Modelos

Deve-se obrigatoriamente escolher e treinar 3 modelos distintos, podendo optar por combinações de:

- Modelos Clássicos: SVM (Support Vector Machines), Random Forest, KNN (K-Nearest Neighbors), XGBoost / Gradient Boosting, Regressão Logística Multinomial, etc.
- Redes Neurais: Perceptron Multicamadas (MLP via Scikit-Learn ou TensorFlow/Keras).
- Cada modelo deve possuir ajuste justificado de ao menos 2 hiperparâmetros (ex: *n_estimators* e *max_depth* na Random Forest; *n_neighbors* e *weights* no KNN; *C* e *kernel* no SVM; número de neurônios, taxa de aprendizado ou funções de ativação na Rede Neural).
- Caso queiram usar outros modelos não citados acima, é permitido, desde que justificadas as escolhas.

Fase 4: Avaliação Comparativa de Desempenho

Para cada um dos 3 modelos avaliados no conjunto de teste independente, apresentar:

- Matriz de Confusão completa (10 x 10) com mapa de calor (heatmap).
- Tabela comparativa consolidada contendo:
 - Acurácia Global (Accuracy)
 - Precisão Média Ponderada (Precision)
 - Revocação/Sensibilidade Média Ponderada (Recall)
 - F1-Score Ponderado (F1-Score)
- Sugestão: [classification_report](#)
- Célula de conclusão técnica apontando: qual dígito apresentou a maior taxa de confusão (ex: 4 vs 9, 3 vs 5, 7 vs 1), qual modelo obteve a melhor performance e qual o impacto do custo computacional (tempo de treino vs acurácia).

Fase 5: Nos próximos sub itens da Fase 5, vocês irão testar a capacidade dos modelos de lidar com cenários fora do padrão ideal de laboratório:

Fase 5.1 - Desafio (A) Treinamento Restrito com Classes Ocultadas (Class Masking)

- Selecione ao menos duas classes para serem totalmente removidas da base de treinamento (por exemplo: ocultar os dígitos 4 e 7).
- Treine um dos seus modelos (clássico ou rede neural) sem nunca ter visto esses dois dígitos durante o ajuste dos pesos.

Fase 5.2 - Desafio (B) Teste de Generalização Extrema (Inferência OOD)

- Submeta o modelo treinado no Desafio A a um conjunto de teste contendo exclusivamente as classes que foram ocultadas (no exemplo: apenas imagens dos dígitos 4 e 7).
- Análise e plote o comportamento do modelo:
 - Como o classificador reage ao ser forçado a classificar algo que nunca viu?
 - Quais classes conhecidas ele atribui a esses dígitos desconhecidos?

- Exiba a matriz de confusão correspondente, apresente suas impressões sobre o teste e discuta o conceito de "falsa certeza" (overconfidence) em modelos de IA.

Fase 5.3 - Desafio (C) Inferência com Imagens Manuscritas Próprias

- Deve-se escrever manualmente um dígito em folha de papel branco com caneta escura (ou desenhar no Paint/GIMP com fundo preto e traço branco).
 - Note que dependendo de sua escolha o pipeline de pré-processamento vai ser diferente.
- Digitalize / fotografe as imagens e construa um pipeline de pré-processamento em Python (usando por exemplo [PIL/OpenCV](#)) contendo:
 - Conversão para escala de cinza;
 - Inversão de cores (caso o papel seja branco com traço preto);
 - Redimensionamento para 28 x 28 pixels com centralização de massa/bounding box;
 - Normalização para o intervalo [0.0, 1.0].
- Realize a predição das suas imagens próprias utilizando o melhor modelo desenvolvido e plote a imagem processada ao lado do gráfico de probabilidades de saída.

5 ROTEIRO DA APLICAÇÃO

Abaixo segue o detalhamento de como a solução de Machine Learning deve ser estruturada, organizada e entregue no repositório do projeto.

5.1 FORMATO DO SISTEMA

A aplicação é um Pipeline de Ciência de Dados ponta a ponta construído em Python. O repositório do projeto deve seguir a estrutura e organização de entrega:

- **Entrega (.ipynb ou .py):** aceite como notebook (.ipynb) ou Python puro (.py). Você pode estruturar seu código em um arquivo único ou segmentá-lo por funções/módulos, se assim desejar.
- **Código Limpo e Explicativo:** organizado com cabeçalhos em Markdown estruturando todas as Fases e Desafios, justificando as tomadas de decisões técnicas.
- **Reprodutibilidade (requirements.txt):** O repositório deve conter um arquivo de texto listando todas as dependências de bibliotecas Python utilizadas (como [pandas](#), [scikit-learn](#), [seaborn](#), [tensorflow](#), [keras](#) e suas respectivas versões), permitindo que qualquer outro cientista de dados consiga recriar o ambiente e rodar o pipeline sem erros.
- **Caminhos Relativos de Dados:** Se os dados forem baixados e lidos localmente no repositório (em vez de carregados por URL direta), eles devem ser organizados em uma pasta de dados específica (ex: [/data](#)). É proibido o uso de caminhos absolutos que apontem para diretórios locais da sua máquina (ex: [C:/Usuarios/SeuNome/Documents/...](#)), pois isso impede a execução do código pelo avaliador. Não esqueçam de organizar as próprias imagens digitalizadas utilizadas no Desafio C.

5.2 DOCUMENTAÇÃO NO README.MD

Crie um arquivo `readme.md` no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas, além do escopo do projeto e como o usuário pode executar o seu sistema.

Algumas dicas interessantes para utilizar na criação do seu portfólio são:

- Criar um nome para o seu software;
- Descrever qual o problema ele resolve;
- Descrever quais técnicas e tecnologias utilizadas. Aqui você também pode inserir alguma imagem ou diagrama para melhor entendimento;
- Descrever como executar;
- Descrever quais melhorias podem ser aplicadas;
- Entre outras coisas.

5.3 USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir *commits* com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por *commit*.
 - Prefira mensagens como “implementa X” ou “corrige Y”, em vez de “X foi implementado” ou “Y foi corrigido”.
- Crie uma *branch* “develop” para concentrar os merges enquanto estiver desenvolvendo as funcionalidades.
- Utilizar *feature branches* a partir da *develop* para cada tarefa.
 - Não excluir a branch após o pull request concluído.
- O código final do projeto deve estar todo “mergeado” no *branch* “main”.

5.4 GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo **máximo de 10 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?
- Argumentação clara, baseada nos conteúdos abordados, de suas escolhas realizadas nesse projeto.

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link, e compartilhe o mesmo

na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no readme.md do seu projeto no repositório do GitHub.

Note: Não devem ser utilizadas qualquer ferramenta de criação de vídeo com IA nessa etapa.

6 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 60% sobre a avaliação do módulo.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto (Nota máxima no bloco: 2,0)

Nº	Critério de Avaliação	0	2,0
1	Realizou a gravação de um vídeo?	NÃO REALIZADO. Não foi realizada a gravação do vídeo.	TOTALMENTE. Gravou o vídeo e abordou todos os tópicos listados no item 5.4

Uso adequado do GitHub e Readme.md (Nota máxima no bloco: 2,0)

Nº	Critério de Avaliação	0	0,5	1.0
2	Foi aberto um branch separado para cada etapa, com mensagens do commit claras e adequadas?	NÃO REALIZADO. Os commits foram realizados diretamente na master/main, e não foi usado mensagens claras.	PARCIALMENTE. Foram abertas branches para cada funcionalidade, porém foram realizados commits de mesma funcionalidade em branches diferentes, ou mensagens inadequadas.	TOTALMENTE. Foram abertas branches para cada funcionalidades, e os commits foram realizados de forma concisa com mensagens adequadas.
3	Criou uma documentação com readme.md?	NÃO REALIZADO. Não criou a documentação.	PARCIALMENTE. Criou uma documentação, porém de forma muito simplificada.	TOTALMENTE. Criou uma documentação completa com ao menos todos os tópicos sugeridos no item 5.2

Desenvolvimento adequado da Aplicação (Nota máxima no bloco: 6,0)

Nº	Critério de Avaliação	0	0.5	1.0
4	Fase 1: Carregamento e Análise Exploratória de Imagens (EDA)	NÃO REALIZADO. Não carregou a base ou não plotou a grade de imagens.	PARCIALMENTE. Plotou as imagens, mas sem análise da estrutura vetorial/pixels.	TOTALMENTE. Exibiu dimensões, distribuição das classes, grade visual e justificou tecnicamente.

5	Fase 2: Pipeline de Pré-processamento e Divisão dos Dados	NÃO REALIZADO. Omitiu o split estratificado ou treinou com dados sem normalizar.	PARCIALMENTE. Fez o split sem estratificação ou normalizou incorretamente.	TOTALMENTE. Realizou a divisão estratificada dos dados em conjuntos de treino e teste, aplicou a normalização adequada (por meio de divisão por 255.0 ou uso de <i>scaler</i>) e apresentou a justificativa técnica para as escolhas adotadas.
6	Fase 3: Implementação e Treinamento dos 3 Modelos	NÃO REALIZADO. Treinou menos de 3 modelos ou modelos sem ajuste de parâmetros.	PARCIALMENTE. Treinou os 3 modelos, mas sem variar hiperparâmetros ou sem justificativa técnica.	TOTALMENTE. Implementou 3 modelos distintos com ajuste claro de ao menos 2 hiperparâmetros cada e justificou tecnicamente.
7	Fase 4: Avaliação Comparativa Completa	NÃO REALIZADO. Não gerou matrizes de confusão ou omitiu métricas.	PARCIALMENTE. Calculou apenas acurácia isolada, sem Precision/Recall/F1 ou sem conclusões.	TOTALMENTE. Gerou as matrizes de confusão para os 3 modelos de Machine Learning analisados, estruturou uma tabela comparativa contendo todas as métricas de desempenho correspondentes e apresentou a justificativa técnica para os resultados obtidos.
8	Desafios A e B: Robustez e Generalização OOD	NÃO REALIZADO. Não implementou o mascaramento de classes ou teste OOD.	PARCIALMENTE. Ocultou as classes, mas não analisou a matriz de confusão dos dígitos não vistos.	TOTALMENTE. Treinou modelo com classes ocultas, testou em OOD, plotou matriz de confusão e justificou tecnicamente..
9	Desafio C: Inferência com Imagens Próprias	NÃO REALIZADO. Não realizou o teste com fotos próprias manuscritas.	PARCIALMENTE. Testou imagens próprias sem pipeline adequado de redimensionamento/centralização.	TOTALMENTE. Construiu o pipeline completo de pré-processamento de imagens redimensionando-as para 28x28 pixels, aplicando as técnicas de centralização e inversão, realizou a etapa de predição do modelo e plotou graficamente as probabilidades resultantes.

7 PLANO DE PROJETO

Ao construir a aplicação proposta, o estudante desenvolve as seguintes competências técnicas:

- Gerenciamento de repositórios de código utilizando Git e GitHub por meio do fluxo de ramificações e histórico de modificações.
- Execução de pipelines de dados em Python, mitigando ocorrências de vazamento de dados (*Data Leakage*) e assimetria estatística.
- Pré-processamento e vetorização de matrizes de imagens digitais para pipelines de aprendizado de máquina.
- Seleção criteriosa entre algoritmos determinísticos/estatísticos clássicos e modelos de redes neurais artificiais.
- Avaliação diagnóstica de classificadores multiclasse por meio de matrizes de confusão e métricas de sensibilidade/precisão.
- Diagnóstico de vulnerabilidades de modelos de aprendizado quando confrontados com dados fora de distribuição (Out-of-Distribution) e ruídos do mundo real.

8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL

É proibida a geração integral e não supervisionada do código por ferramentas geradoras de código. O estudante deve ser o autor do código e compreender cada linha submetida. O uso de ferramentas de IA é permitido exclusivamente como suporte de estudos conceituais, depuração de erros de sintaxe ou esclarecimento de dúvidas teóricas sobre a documentação das bibliotecas. O projeto será arguido com base no domínio técnico demonstrado no vídeo e no repositório.